Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н.Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4.2

по курсу «Методы оптимизации»

«Метод простых симплексов и Нелдера-Мида в 3D»

Студент группы ИУ9-82Б Шемякин В.А.

Преподаватель Посевин Д. П.

Москва 2026

1 Задание

Сравнение Простых симплексов и метода Нелдера-Мида, но целевая функция зависит от 3 переменных, отобразить два набора последовательностей тэтраэдров.

Протестировать данные методы на примере функций Швеффеля, Растригина и Розенброкка зависящих он трех координат.

Набор многогранников формирующих минимизирующую последовательность отобразить в трехмерном пространстве, а значение целевой функции цветом. Значение целевой функции цветом показывается так: в начальной точке минимизирующей последовательности цвет многогранника (симплекса) светлый, а при приближении к минимуму цвет нового многогранника (симплекса) становится более темным.

2 Результаты

Исходный код программы представлен в листинге 1.

```
using LinearAlgebra
using Statistics
using Plots
using Colors
Plots.default(background=:white, foreground_color=:darkgray)

function initial_simplex1(n)
    P = (1 / (n*sqrt(2))) * (sqrt(n + 1) + n - 1)
    Q = (1 / (n*sqrt(2))) * (sqrt(n + 1) - 1)
    X = zeros(n, n + 1)
    for k in 2:n+1
        for j in 1:n
            X[j, k] = (j == k - 1) ? P : Q
        end
    end
    return X
end

function initial_simplex2(n)
    R = [sqrt(1 / (2 * i*(i + 1))) for i in 1:n]
    V = [sqrt(i / (2 * (i + 2))) for i in 1:n]
```

```
    X = zeros(n, n + 1)
    for j in 1:n
        X[j, 1] = -R[j]
    end
    for k in 2:n+1
        for j in 1:n
            X[j, k] = (j == k - 1) ? V[k-1] : -R[j]
        end
    end
    return X
end

function simple_simplex(f, x0;    =1e-6, maxiter=10000,    =0.5, history=nothing, init_method=1, ref_max_ratio=2.0)
    n = length(x0)
    S = (init_method == 1) ? initial_simplex1(n) : initial_simplex2(n)
    X = x0 .+ S
    fvals = [f(X[:, i]) for i in 1:n+1]
    iter = 0
    if history !== nothing
        push!(history, copy(X))
    end
    while iter < maxiter
        order = sortperm(fvals)
        fvals = fvals[order]
        X = X[:, order]
        x_centroid = sum(X[:, 1:n], dims=2) / n
        x_ref = 2 * x_centroid - X[:, n+1]
        d_worst = norm(X[:, n+1] - x_centroid)
        d_ref = norm(x_ref - x_centroid)
        if d_worst > 1e-12 && d_ref > ref_max_ratio * d_worst
            x_ref = x_centroid + (x_ref - x_centroid) .* (ref_max_ratio * d_worst / d_ref)
        end
        f_ref = f(x_ref)
        if f_ref < fvals[n+1]
            X[:, n+1] = x_ref
            fvals[n+1] = f_ref
        else
            for i in 2:n+1
                X[:, i] = X[:, 1] +     * (X[:, i] - X[:, 1])
                fvals[i] = f(X[:, i])
            end
        end
```

```
        if history !== nothing
            push!(history, copy(X))
        end
        if sqrt(sum(sum((X[:, i] .- x_centroid).^2) for i in 1:n+1) / (n+1)) <
            break
        end
        iter += 1
    end
    return X[:, 1], fvals[1], iter
end

function nelder_mead(f, x0;    =1.0,    =2.0,    =0.5,    =0.5,    =1e-6, maxiter=10000, history=nothing, init_method=1)
    n = length(x0)
    S = (init_method == 1) ? initial_simplex1(n) : initial_simplex2(n)
    X = x0 .+ S
    fvals = [f(X[:, i]) for i in 1:n+1]
    iter = 0
    if history !== nothing
        push!(history, copy(X))
    end
    while iter < maxiter
        order = sortperm(fvals)
        fvals = fvals[order]
        X = X[:, order]
        x_best = X[:, 1]
        x_worst = X[:, n+1]
        x_centroid = sum(X[:, 1:n], dims=2) / n
        f_best = fvals[1]
        f_worst = fvals[n+1]
        f_second_worst = fvals[n]
        x_ref = x_centroid .+     .* (x_centroid .- x_worst)
        f_ref = f(x_ref)
        if f_ref < f_second_worst
            if f_ref < f_best
                x_exp = x_centroid .+     .* (x_ref .- x_centroid)
                f_exp = f(x_exp)
                if f_exp < f_ref
                    X[:, n+1] = x_exp
                    fvals[n+1] = f_exp
                else
                    X[:, n+1] = x_ref
                    fvals[n+1] = f_ref
```

```
                end
            else
                X[:, n+1] = x_ref
                fvals[n+1] = f_ref
            end
        else
            if f_ref < f_worst
                x_oc = x_centroid .+     .* (x_ref .- x_centroid)
                f_oc = f(x_oc)
                if f_oc      f_ref
                    X[:, n+1] = x_oc
                    fvals[n+1] = f_oc
                else
                    for i in 2:n+1
                        X[:, i] = x_best .+     .* (X[:, i] .- x_best)
                        fvals[i] = f(X[:, i])
                    end
                end
            else
                x_ic = x_centroid .-     .* (x_centroid .- x_worst)
                f_ic = f(x_ic)
                if f_ic < f_worst
                    X[:, n+1] = x_ic
                    fvals[n+1] = f_ic
                else
                    for i in 2:n+1
                        X[:, i] = x_best .+     .* (X[:, i] .- x_best)
                        fvals[i] = f(X[:, i])
                    end
                end
            end
        end
        if history !== nothing
            push!(history, copy(X))
        end
        if sqrt(sum((fvals[i] - sum(fvals)/(n+1))^2 for i in 1:n+1) / (n+1)) <
            break
        end
        iter += 1
    end
    return X[:, 1], fvals[1], iter
end
```

```

schwefel(x) = 418.9829 * length(x) - sum(x .* sin.(sqrt.(abs.(x))))
rastrigin(x) = 10 * length(x) + sum(x.^2 .- 10 .* cos.(2    .* x))
rosenbrock(x) = sum(100 * (x[i+1] - x[i]^2)^2 + (1 - x[i])^2 for i in 1:length(x)-1)

const TETRA_EDGES = [(1,2), (1,3), (1,4), (2,3), (2,4), (3,4)]

function plot_tetrahedron_history_3d_one(f, history, title_str, filename;
                                         max_simplices=40, use_green=true, max_full=150)
    N = length(history)
    idx = N <= max_full ? (1:N) : round.(Int, range(1, N, length=max_simplices))
    hist = [history[i] for i in idx]
    all_x = Float64[]
    all_y = Float64[]
    all_z = Float64[]
    for X in hist
        for j in 1:4
            push!(all_x, X[1, j])
            push!(all_y, X[2, j])
            push!(all_z, X[3, j])
        end
    end
    f_all = [mean(f(X[:, j]) for j in 1:4) for X in hist]
    f_min = minimum(f_all)
    f_max = maximum(f_all)
    rng = max(f_max - f_min, 1e-12)
    span_x = max(maximum(all_x) - minimum(all_x), 1e-10)
    span_y = max(maximum(all_y) - minimum(all_y), 1e-10)
    span_z = max(maximum(all_z) - minimum(all_z), 1e-10)
    margin = 0.2 * max(span_x, span_y, span_z)
    xmin = minimum(all_x) - margin
    xmax = maximum(all_x) + margin
    ymin = minimum(all_y) - margin
    ymax = maximum(all_y) + margin
    zmin = minimum(all_z) - margin
    zmax = maximum(all_z) + margin
    plt = plot(legend=false, xlabel=" x  ", ylabel=" x  ", zlabel=" x  ", title=title_str,
               xlims=(xmin, xmax), ylims=(ymin, ymax), zlims=(zmin, zmax), background=:white)
    for X in hist
```

```
        f_simp = mean(f(X[:, j]) for j in 1:4)
        t = (f_simp - f_min) / rng
        intensity = 0.25 + 0.75 * t
        clr = use_green ? RGB(0.2, 0.2 + 0.8 * intensity, 0.2) : RGB(0.2, 0.2, 0.2 + 0.8 * intensity)
        for (i, j) in TETRA_EDGES
            plot!(plt, [X[1, i], X[1, j]], [X[2, i], X[2, j]], [X[3, i], X[3, j]],
                  seriestype=:path3d, color=clr, linewidth=0.6, linestyle=:solid)
        end
    end
    savefig(plt, filename)
    return plt
end

n = 3
x0_schwefel = [400.0, 400.0, 420.0]
x0_rastrigin = [3.0, 3.0, 2.0]
x0_rosenbrock = [-1.5, 2.0, 0.5]

for (name, fname, f, x0) in [
    ("                    ", "schwefel", schwefel, x0_schwefel),
    ("                      ", "rastrigin", rastrigin, x0_rastrigin),
    ("                      ", "rosenbrock", rosenbrock, x0_rosenbrock)
]
    println("\n$name")
    hist_simple = []
    hist_nelder = []
    x_s, f_s, it_s = simple_simplex(f, x0, history=hist_simple)
    x_n, f_n, it_n = nelder_mead(f, x0, history=hist_nelder)
    println("                                    :  x* = $x_s,  f* = $f_s,                    = $it_s")
    println("                -          :       x* = $x_n,  f* = $f_n,                    = $it_n")
    plot_tetrahedron_history_3d_one(f, hist_simple, "$name                                    ",
        "main2_$(fname)_simple_3d.png"; use_green=true)
    plot_tetrahedron_history_3d_one(f, hist_nelder, "$name                -          ",
        "main2_$(fname)_nelder_3d.png"; use_green=false)
end
```

Результат представлен на рисунках 1– 3.

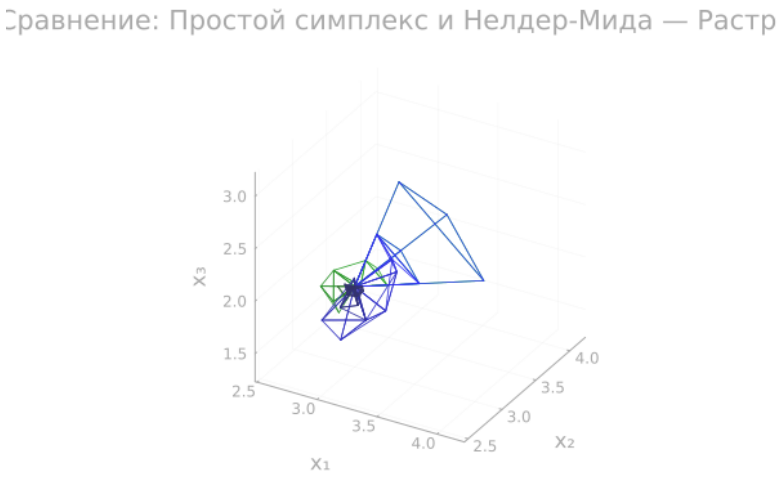

Рис. 1 — Результат

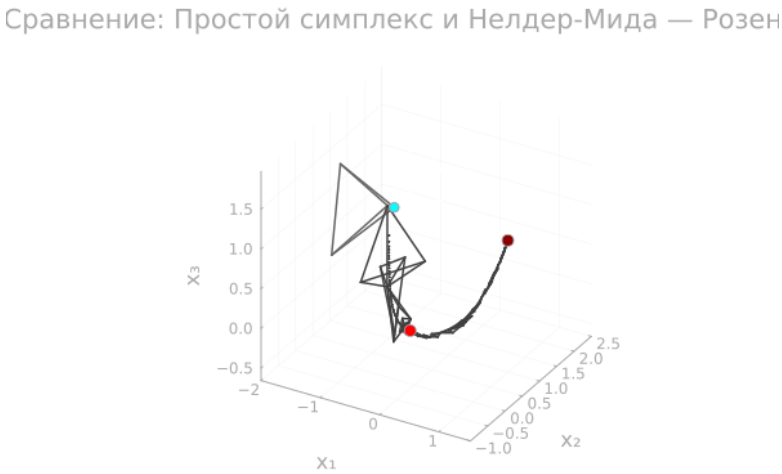

Рис. 2 — Результат

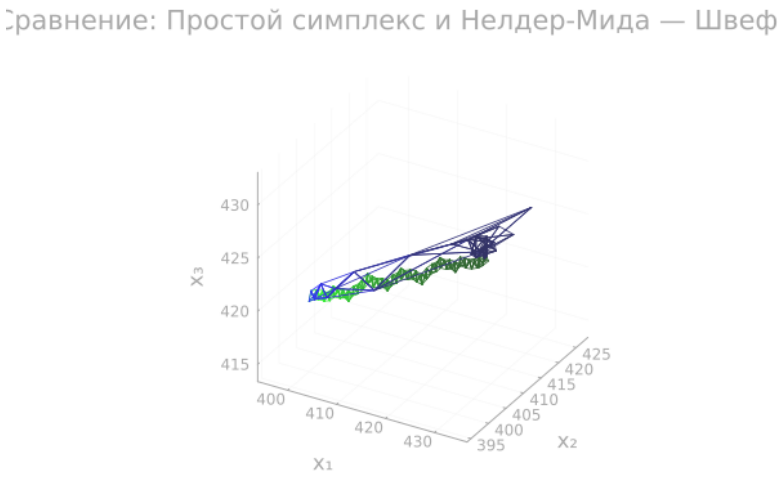

Рис. 3 — Результат